

Exercise 1: Message Queues

Two processes: client and server, communicate via two message queues " WORDS " and "VOWELS".

1. The client reads a message from the standard input and sends it to the server through one of two message queues: WORDS or VOWELS. If the client sends the message through the WORDS queue, the server reads the message and calculates the number of words in the sentence, then sends the result back to the client. If the client sends the message through the VOWELS queue, the server reads the message and calculates the number of vowels in the sentence, then sends the result back to the client. For example, if the client sends the message "operating systems" through the WORDS queue, the server returns 2, and if the same message is sent through the VOWELS queue, the server returns 6. When the client receives the result from the server, it prints it out. You may assume the maximum size of any message is 256 bytes. (N.B. the two counting functions are done for you at the end of the document)
2. Multiple clients must be able to connect to the words and vowels queues. However, what happens if one client puts a message to process on the system and another client is waiting for its response from the queue? There are different ways to handle this, but you should handle this using "typed messages". Each client has a particular client number based on the last 4 digits of its process ID. Use this 4-digit number as the client identifier and make it the message type so that each client can always be sure to receive the message that it is waiting on.
3. The server and client should be running forever using a while loop; however you can terminate the program using ctrl+c (SIGINT). In the server, when you terminate the program, you should delete the message queues before termination using the SIGINT handler. Write two C programs client.c and server.c that implement the behavior explained above.

Clarification:

- When the user presses Ctrl+C in the client program, this means that you should terminate the client program ONLY. (this does not mean you destroy the message queue).
- When the user presses Ctrl+C in the server program, this means that you should terminate the server program ONLY. (here, you need to destroy the message queue before termination).
- When the server removes the message queue, the client will automatically fail to send anything on this queue (because it is already destroyed). This is completely normal.

N.B.: You can run the server and multiple clients at the same time using different terminals.

Exercise 2: Shared Memory & Semaphores

Two processes, client and server, communicate via semaphores and a shared memory segment.

1. The client reads a message from the standard input and writes it to the shared memory. Then it signals the server asking it to process the data it wrote to the shared memory. The server is specialized in counting either the number of words or the number of vowels in the message, depending on the request written by the client. Therefore, if the client writes the message "operating systems" in the shared memory and requests WORDS, the server will read it, count the number of words, and write 2 to the shared memory. If the client requests VOWELS, the server will read the same message, count the number of vowels, and write 6 to the shared memory. Then it signals the client telling it that it finished processing the data. Write two C programs client.c and server.c that implement the behavior explained above.

**Computer Engineering Dept.
Cairo University
Operating Systems
IPC**

Deliverables:

1. **FirstName_SecondName_ID.zip** containing two directories as follows:
 - a. Exercise1 (contains source code for Exercise 1)
 - i. client.c
 - ii. server.c
 - b. Exercise2 (contains source code for Exercise 2)
 - i. client.c
 - ii. server.c
 - c. All other files that you used for each exercise that will allow us to run your code on our side. (Place these files in each directory).
 - d. A video containing the following cases for BOTH requirements:
 - i. 4 terminals, one for the server and three for the clients. Two of the three clients will each send a different sentence to the server to count the WORDS. The third client will send a third sentence to the server to count the VOWELS. The server should be running before the clients send anything. The server should sleep at the beginning for 10 seconds (enough for you to run the three client scripts and send the request from each one).
 - ii. 4 terminals, one for the server and three for the clients. Two of the three clients will each send a different sentence to the server to count the WORDS. The third client will send a third sentence to the server to count the VOWELS. The server should be running before the clients send anything. The server should sleep at the beginning for 10 seconds, enough for you to run the three client scripts and send the request from the two clients targeting WORDS. After the server processes each request, it should sleep 3 seconds and while that is happening, send the request from the third client (the one targeting VOWELS)
 - iii. Following either of the previous two cases, terminate one client script, and send one request from each of the other two clients. Finally, terminate the server script, and try sending a request from either of the two clients (if possible)

Computer Engineering Dept.
Cairo University
Operating Systems
IPC

Counting Functions:

Code for counting vowels:

```
int count_vowels(const char *str)
{
    int count = 0;

    while (*str)
    {
        char c = *str;

        if (c=='a' || c=='e' || c=='i' || c=='o' || c=='u' ||
            c=='A' || c=='E' || c=='I' || c=='O' || c=='U')
        {
            count++;
        }

        str++;
    }

    return count;
}
```

Code for counting words:

```
int count_words(const char *str)
{
    int count = 0;
    int in_word = 0;

    while (*str)
    {
        if (*str != ' ' && in_word == 0)
        {
            count++;
            in_word = 1;
        }
        else if (*str == ' ')
        {
            in_word = 0;
        }

        str++;
    }

    return count;
}
```